



HARNESS TOOLKIT · TECH LEADS CLUB

Autonomia não se pede. Se constrói.

PROMPT É A CONVERSA.
HARNESS É A EXECUÇÃO.

A camada de QA e governança que falta quando você dá autonomia real a um agente de código, dentro do **Cursor** e do **Claude Code**. **8 regras que configuração nenhuma alcança, 3 checagens sempre ativas e 24 rails que o time liga.**

Felipe Rodrigues

Staff AI Engineer · 12.09.2026



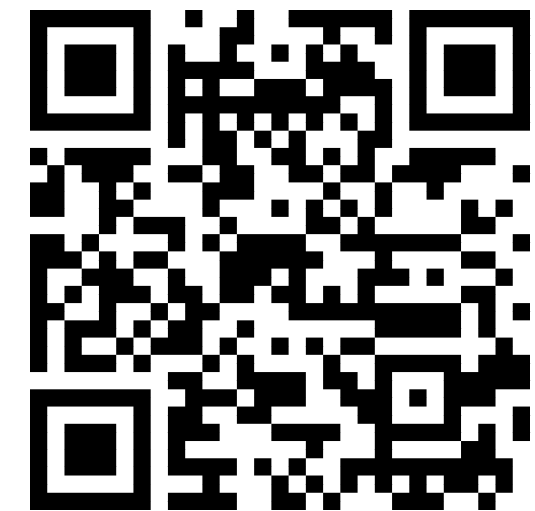
SOBRE MIM

Felipe Rodrigues

Staff AI Engineer @ BHub.ai

AWS Community Builder · SpaceXAI Ambassador · Mentor @ Tech Leads Club

Criei a **tlc-spec-driven** e o **harness-toolkit**, que eu mantenho hoje junto com a Tech Leads Club. Os dois nasceram do mesmo problema: agente que trabalha sozinho precisa de alguma coisa além de um prompt bem escrito.



Felipe Rodrigues

Staff AI Engineer

[linkedin.com/in/felipfr](https://www.linkedin.com/in/felipfr)



O PROBLEMA É ESTRUTURAL, NÃO DE QUALIDADE DO MODELO

Um agente com autonomia real não conversa com você, ele **executa**

SHELL

Comando destrutivo fora do projeto.
Credencial copiada para dentro do transcript.

ARQUIVOS

Teste apagado para o gate ficar verde.
A própria política do harness reescrita.

GIT

Histórico reescrito com **--force**. PR aberto sem ninguém ter revisado.

SUBAGENTES

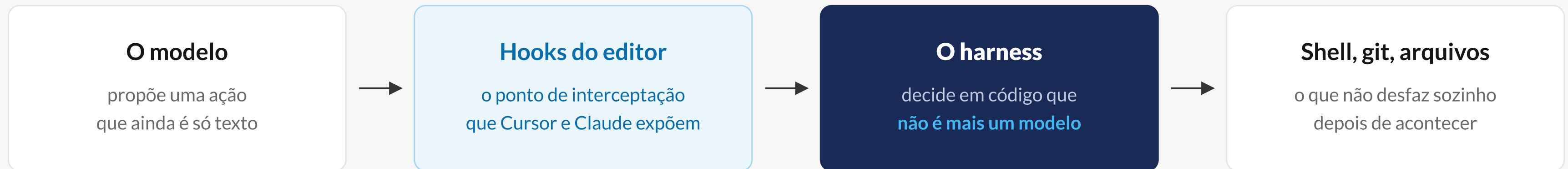
Outro agente, outro modelo, sem limite herdado de quem o criou.

Prompt é conversa, não controle de execução. A pergunta que importa não é se o agente é bom, e sim o que existe entre a intenção dele e essas quatro superfícies quando ele erra.



DEFINIÇÃO

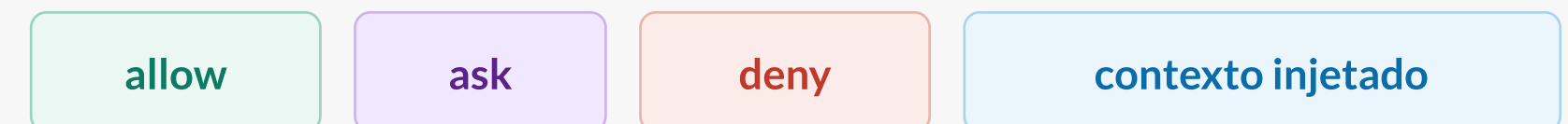
Uma camada de execução entre o modelo e o mundo real



Não é plugin de prompt nem opinião de um segundo modelo.

É infraestrutura: mesma entrada, mesma decisão, sempre.

AS QUATRO RESPOSTAS POSSÍVEIS

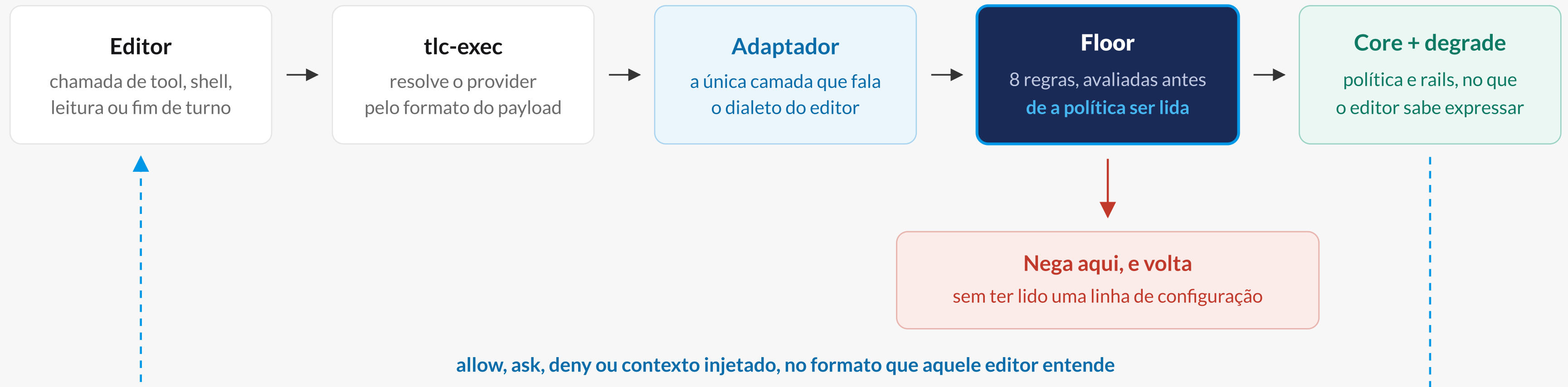


Toda ação recebe uma delas, e fica registrada.



UM EVENTO, DO INÍCIO AO FIM

O floor decide **antes** de qualquer configuração ser lida



Uma regra do floor não pode ser desligada por configuração porque ela roda antes de a configuração existir no fluxo.



Dois editores, uma lógica de decisão, checada no build



Não é convenção: um verificador reprova o build se uma dessas setas vermelhas aparecer no código.
E uma varredura proíbe as palavras cursor, claude e anthropic em qualquer lugar sob core/ e contracts/.

Suportar um terceiro editor é escrever um adaptador novo, não reabrir a lógica de decisão.



O PISO DA EXECUÇÃO

Oito regras que configuração nenhuma alcança

REGRA	NEGA	MAS PERMITE
secret-access	Ler .env, ~/.ssh, ~/.aws, *.pem, pelo shell ou pela tool de leitura do próprio editor	Procurar o endereço literal nos arquivos locais: grep não faz requisição
unprovable-execution	Baixar um programa da rede e entregar ao shell, por pipe, substituição ou eval	Um fetch sem shell depois, e um shell alimentado por um arquivo local que dá para ler antes
policy-surface-write	Toda rota que o agente tem até a política e o estado do próprio harness	Ler, com cat, head, grep, jq, ls, stat, test
wiring-tamper	Sobrescrever o arquivo em que o editor registra os hooks, o que silenciaria o harness inteiro de uma vez	Ler o mesmo caminho por um leitor provado
history-rewrite	git push --force	--force-with-lease, que já recusa sozinho quando o remoto andou

O PONTO Cada regra nega a cauda destrutiva e deixa passar o caminho legítimo que costuma ser confundido com ela. Uma regra do floor não tem chave de desligar. O arquivo que a desligaria é justamente um dos que o agente não alcança.



ACIMA DO PISO

Vinte e quatro interruptores, e o time decide **onde aperta**

RAIL	O QUE ELE FAZ
Grind	Roda o seu lint e o seu test contra os arquivos que o turno mudou, e devolve o agente ao trabalho até passar
Ship gate	Confere um "terminei" contra evidência recente de PASS para os caminhos que aquele turno tocou
Comment gate	Barra o comentário narrativo que o turno adicionou. O commit de onde ele partiu nunca é reavaliado
Duplication gate	Barra o trecho que o turno acrescentou e que já existia em outro lugar do projeto
Subagent allowlist	Restringe o modelo que um subagente pode usar, e bloqueia as variantes -fast por padrão
Lessons	Registra o que uma falha repetida ensinou e reinjeta isso na próxima sessão e no retry seguinte

O PONTO Todo rail é on/off e cada um foi apresentado com o benefício **e** o custo dele no assistente de instalação. **Com zero rails ligados você ainda tem o floor e as três checagens sempre ativas.**



JULGAMENTO HUMANO NO LOOP

Postura não muda o rigor, muda o que chega até você

PAIRED

Pergunta antes de agir

Qualquer movimento relevante, inclusive um comando que alcança a rede ou que pode sobrescrever um caminho que já existe.

SOLO · PADRÃO

Três coisas sobem até você

Ação irreversível ou destrutiva, beco sem saída real depois de esgotar as fontes, e ambiguidade que muda o resultado.

FOCUS

Só o que não tem volta

Ação destrutiva ou beco sem saída. A ambiguidade o agente resolve sozinho, e declara a suposição que adotou.

POR QUE ISSO IMPORTA **A barra de evidência é idêntica nas três.** Os mesmos gates, os mesmos critérios de pronto. O que muda é só quando uma decisão sobe para você em vez de ser tomada e registrada. E o registro é auditável: **tlc harness obs report** e **tlc harness attest**, encadeado por hash.



Avaliado contra uma taxonomia pública, com as lacunas junto

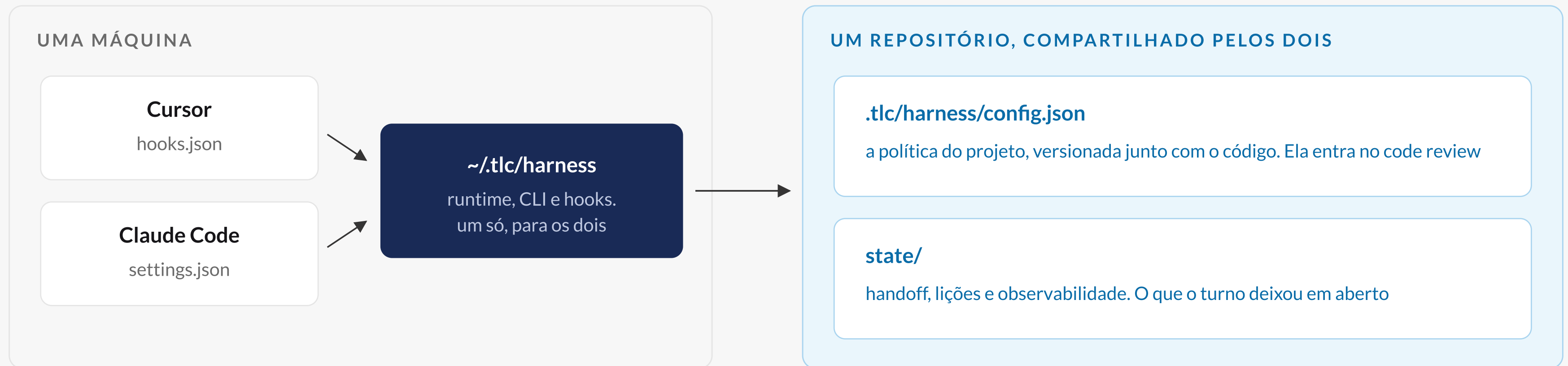
RISCO	ESTADO	O QUE FICA ABERTO
ASI02 Uso indevido de ferramentas	coberto	nada
ASI05 Execução de código inesperada	coberto	nada
ASI08 Falhas em cascata do agente	coberto	nada
ASI09 Exploração da confiança humano-agente	coberto	nada
ASI01 Sequestro do objetivo do agente	parcial	Comando reescrito antes de rodar não é rastreado até o conteúdo que o originou
ASI03 Abuso de identidade e privilégio	parcial	A identidade é a sessão do editor, não há credencial por agente para amarrar a decisão
ASI04 Cadeia de suprimentos agêntica	parcial	Advisory, licença e typosquat exigem rede a cada parada, e ficam para um comando de gate
ASI06 Envenenamento de memória e contexto	parcial	O repositório global de lições é escrito por sessões de outros projetos, e não dá para selar por projeto
ASI10 Agentes desonestos	parcial	Não há baseline de comportamento, então um agente errando de forma plausível não é sinalizado
ASI07 Comunicação insegura entre agentes	não aplicável	Subagente é criado pelo host e não troca mensagem que o harness intermedeie

O PLACAR INTEIRO **4 cobertos • 5 parciais • 1 não aplicável.** Publicar o parcial com o motivo do parcial é o que dá para conferir. Uma ferramenta que se declara 10 de 10 contra uma taxonomia de risco está vendendo, não medindo.



DUAS CAMADAS

Trocar de editor não é **reescrever a governança**



Duas pessoas no mesmo repositório, uma no Cursor e outra no Claude Code, enxergam um handoff só.



CUSTO DE ADOÇÃO

Uma linha, e o floor já vale em **todo repositório**

```
$ npm i -g @tech-leads-club/harness-toolkit && tlc harness install
```

01

Reinicie o editor

O **install** descobre qual dos dois você tem e liga só aquele, sem tocar no que já estava lá.

02

Já vale em tudo

O floor e as três checagens sempre ativas valem em **qualquer repositório**, sem configurar nada.

03

Regra própria, se quiser

Diga "**setup harness**" ao agente. Para conferir qualquer coisa, **tlc harness doctor**.

FICHA TÉCNICA

Zero dependências de runtime, TypeScript estrito, procedência SLSA no pacote publicado, e Linux, macOS e Windows testados em CI. **Licença Elastic 2.0.**



Dar autonomia a um agente é uma decisão de **arquitetura**, não de prompt

Se isso te poupar um problema, deixe uma estrelinha.



harness-toolkit

Tech Leads Club

github.com/tech-leads-club